

---

# 2026 年全国大学生电子设计竞赛

## 车载平衡滚球运动控制系统（H 题）（本科组）



---

## 摘要

本文针对 H 题车载平衡滚球运动控制系统，提出底盘主控、球杆控制与车载视觉协同的分层方案。MSPM0G3507 负责红外循迹、轮速闭环、计时和任务管理；树莓派通过固定俯视摄像头检测钢球位置与速度，经有线串口发送至大疆 C 型开发板，并完成图像传输与录像；C 板构建球位置外环和摆杆角度、速度内环，驱动 QD4310 执行机构，同时融合底盘状态与 BMI088 加速度前馈，提升车辆运动中的抗扰能力。系统采用带序号、时间戳和 CRC 校验的通信协议，并设置视觉失联、CAN 异常、球体越界及控制饱和保护。通过视觉标定、独立底盘、静止球控、运动中居中和任意位置控制等分阶段调试，实现小车稳定循迹、钢球快速定位和全程误差约束，为系统可靠实现及竞赛验收提供技术依据。

**关键词：**车载平衡滚球；视觉检测；分层控制；串级闭环；加速度前馈

---

## 目录

|                              |   |
|------------------------------|---|
| 1、系统方案 .....                 | 1 |
| 1.1、设计目标 .....               | 1 |
| 1.2、总体方案 .....               | 1 |
| 1.3、各部分方案比较与选择 .....         | 2 |
| 1.3.1、底盘主控芯片的方案论证与选择 .....   | 2 |
| 1.3.2、钢球视觉处理平台的方案论证与选择 ..... | 3 |
| 1.3.3、摆杆执行机构的方案论证与选择 .....   | 3 |
| 1.3.4、车辆加速度测量方案的论证与选择 .....  | 4 |
| 2、系统理论分析与计算 .....            | 4 |
| 2.1、红外循迹与差速运动原理 .....        | 4 |
| 2.2、钢球视觉识别与坐标换算原理 .....      | 4 |
| 2.3、球位置串级闭环控制原理 .....        | 5 |
| 3、电路与程序设计 .....              | 6 |
| 3.1、电路设计 .....               | 6 |
| 3.1.1、循迹小车总体电路 .....         | 6 |
| 3.1.2、平衡滚球控制电路 .....         | 6 |
| 3.2 设计思路 .....               | 7 |
| 3.2.1 循迹小车程序设计总体思路 .....     | 7 |
| 3.2.2 车载滚球控制装置 .....         | 7 |
| 4.测试方案和测试结果 .....            | 7 |
| 4.1 测试方案 .....               | 7 |
| 4.1.1 循迹部分硬件测试 .....         | 7 |
| 4.1.2 软件仿真测试 .....           | 8 |
| 4.1.3 硬件软件联调 .....           | 8 |
| 4.2 测试条件与仪器 .....            | 8 |
| 4.3 测试结果及分析 .....            | 8 |
| 4.3.1 测试结果（数据） .....         | 8 |

---

|                     |   |
|---------------------|---|
| 4.3.2 测试分析与结论 ..... | 8 |
|---------------------|---|

---

## 车载平衡滚球运动控制系统（H 题）

### 1、系统方案

#### 1.1、设计目标

设计并制作一套车载平衡滚球运动控制系统。系统由自动循迹小车、带凹槽摆杆、钢球、摆杆执行机构、车载视觉装置和计时显示装置组成。小车应在投影不脱离黑色环形轨迹的条件下稳定行驶，摆杆应安装在车体允许范围内，钢球位置必须通过摄像头实时检测，底盘循迹仅使用红外光电传感器完成。

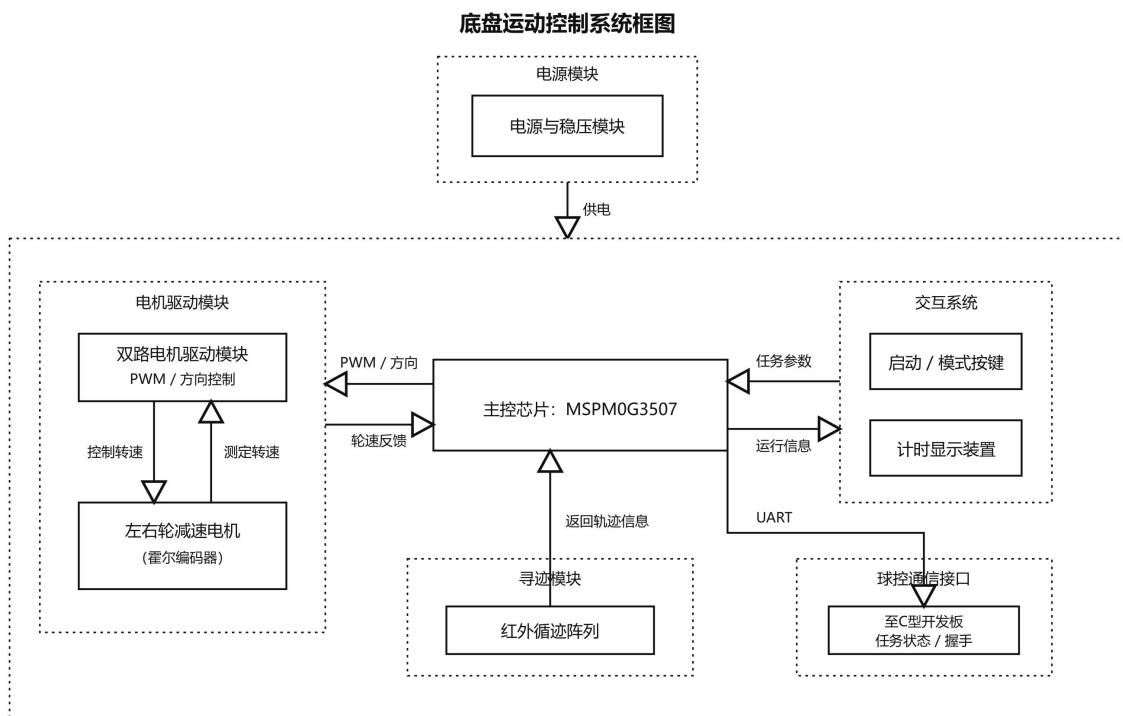
系统需完成以下主要任务：小车在规定时间内循迹一圈并停回起点；静止状态下使钢球按“0 点→+5 cm→-5 cm”的顺序快速移动并稳定；小车由 A 点驶向 B 点及行驶整圈时，使钢球保持在中心或给定位置附近。设计中将球位置最大允许误差控制在 1 cm 以内，并以 0.6~0.8 cm 作为实验室调试目标，同时满足图像实时显示、录像回放、计时、故障保护和重复测试要求。

#### 1.2、总体方案

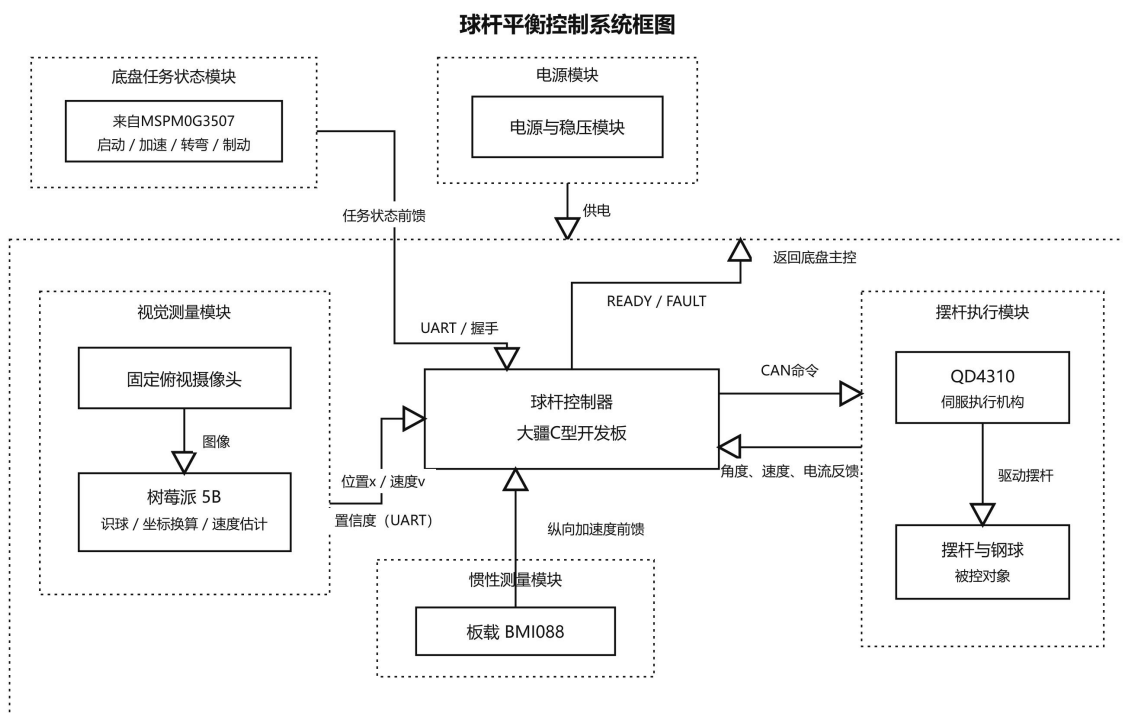
本系统采用“底盘任务主控、球杆专用控制、车载视觉测量”的分层结构。MSPM0G3507 负责循迹、车轮速度控制、比赛计时和停车，并向大疆 C 型开发板发送车辆启动、加速、转弯和制动等任务状态。

俯视摄像头采集钢球图像，树莓派 5B 完成钢球识别、位置计算和速度估计，再通过 UART 将数据发送至 C 板。无线链路只用于视频显示和回放，不参与控制。

C 板根据钢球位置计算摆杆目标角度，通过 CAN 总线控制 QD4310 执行机构。板载 BMI088 测量车体加速度并提供前馈补偿，从而形成“钢球位置外环、摆杆伺服内环、车辆状态前馈”的控制结构。当出现视觉过期、通信超时、钢球越界或传感器异常时，系统立即限制输出或停车，保证运行安全。



图一：底盘运动控制系统框图



图二：球杆平衡控制系统框图

### 1.3、各部分方案比较与选择

#### 1.3.1、底盘主控芯片的方案论证与选择

---

**方案 1: MSPM0G3507。**该器件最高主频为 48 MHz，具有定时器、串口、ADC 及较丰富的 GPIO 资源，能够同时完成红外采样、编码器测速、双轮速度控制、按键计时和任务状态机。底盘功能可脱离视觉与球控模块独立运行，便于分阶段调试和故障隔离。

**方案 2: STM32F103。**该芯片生态成熟、资料丰富，能够满足一般循迹控制需求，但本系统已有 MSPM0G3507 底盘基础，若更换主控需要重新完成底层驱动、定时器分配和整车联调，增加开发工作量与比赛风险。

**方案 3: 由树莓派统一控制。**树莓派计算能力较强，但通用操作系统的调度抖动和启动过程不利于直接承担电机快速闭环与安全停机；视觉处理、图传和底盘控制集中后，任一软件异常都可能同时影响多个子系统。

综合实时性、硬件资源、已有工程基础和模块独立性，底盘主控选择 MSPM0G3507。树莓派只负责视觉和视频，大疆 C 板只负责球杆控制，三者通过明确协议协同。

### 1.3.2、钢球视觉处理平台的方案论证与选择

**方案 1: OpenMV 视觉模块。**OpenMV 体积小、功耗低，适合完成阈值分割和简单几何识别，但在高帧率采集、畸变校正、动态坐标映射、视频编码和同时记录多路调试信息时处理余量有限。

**方案 2: 树莓派 5B。**树莓派具备较强的图像处理能力，可完成去畸变、滤波、亚像素圆心定位、状态估计和视频传输，并能保存时间戳与原始录像。通过有线 UART 发送少量控制数据，可将图像处理与实时电机控制解耦。

因此选择固定俯视摄像头与树莓派 5B 作为车载视觉平台。场外计算机接收视频并录像，钢球位置控制始终使用车上树莓派至 C 板的有线数据链路。

### 1.3.3、摆杆执行机构的方案论证与选择

**方案 1: 舵机。**舵机控制简单、集成度高，但普通舵机存在死区、回差和速度限制，内部控制参数通常不可充分调整，难以获得连续、快速且可观测的角度—速度—电流内环。

**方案 2: 步进电机。**步进电机便于通过脉冲数控制角度，低速保持能力较强，但开环运行时无法直接确认实际角度；在快速换向或负载突变时可能失步，且振动会通过摆杆传递给钢球和摄像头。

**方案 3: QD4310 伺服执行机构。**QD4310 可通过 CAN 接收控制命令并返回角度、速度和电流信息，便于构建摆杆快速内环、设置机械软限位并检测反馈超时或异常电流。其连续反馈更适合与视觉球位置外环组成串级系统。

---

综合响应速度、反馈完整性、可调性和安全保护需求，摆杆执行机构选择 QD4310，并由大疆 C 型开发板通过 CAN 总线控制。

#### 1.3.4、车辆加速度测量方案的论证与选择

**方案 1：车轮编码器差分。**编码器能够反映轮速变化，但差分会放大噪声，且打滑、差速转弯和轮径误差使轮速变化不等同于车体真实纵向加速度，因此只适合低频趋势校验。

**方案 2：外接 JY61P。**外部 IMU 使用方便，但需要新增串口、安装坐标标定和另一套时间同步机制，增加线缆与故障点。

**方案 3：板载 BMI088。**BMI088 与 C 板硬件连接短，可通过 SPI 获得较高频率的惯性数据，并与摆杆控制任务在同一处理器内完成时间对齐。

因此优先采用板载 BMI088。必须完成六面静态标定、坐标变换、零偏和比例校正、温漂检查、低通滤波以及姿态/重力处理后，才能将纵向线加速度用于前馈；未完成上述步骤时仅作观测，不直接参与控制。

## 2、系统理论分析与计算

### 2.1、红外循迹与差速运动原理

红外循迹利用黑、白表面对红外光反射率的差异。红外发射管向地面发光，白色区域反射较强，黑色轨迹吸收较多、反射较弱。控制器读取多个红外接收通道的数字量或模拟量，并根据各通道相对车体中心的位置计算横向偏差。为减小单个传感器跳变造成的转向抖动，可将第  $i$  个传感器状态记为  $s_i$ ，其横向权重记为  $w_i$ ，则循迹偏差为：

$$e_l(k) = \frac{\sum_i w_i s_i(k)}{\sum_i s_i(k)} \quad (1)$$

当全部传感器均未检测到轨迹时，不再直接使用上式，而是根据上一时刻偏差方向执行受限搜索，并在规定时间内未重新捕获轨迹时停车。得到偏差  $e_l$  后，可采用比例—微分或 PID 算法计算转向修正量  $\Delta v$ 。设小车基础线速度为  $v_0$ ，则左右轮目标速度可表示为：

$$v_L^* = v_0 - K_t e_l, \quad v_R^* = v_0 + K_t \quad (2)$$

### 2.2、钢球视觉识别与坐标换算原理

钢球位置检测采用固定俯视相机。

设去畸变图像中摆杆两端标记的像素坐标分别为  $p_A$  和  $p_B$ ，钢球圆心为  $p$ ，摆杆有效长度为  $L$ 。将钢球圆心投影到球道轴线上，可得到无量纲轴向坐标  $\lambda$ ：

$$\mathbf{d} = \mathbf{p}_B - \mathbf{p}_A, \quad \lambda = \frac{(\mathbf{p} - \mathbf{p}_A) \cdot \mathbf{d}}{\|\mathbf{d}\|^2}, \quad x = (\lambda - \lambda_0)L \quad (5)$$

其中  $\lambda_0$  为中心  $O$  点在同一轴线上的归一化坐标， $x$  为相对  $O$  点的实际位置。该方法利用每帧检测到的摆杆标记更新轴线，可减小车体振动和摆杆转动对固定像素映射的影响。若透视畸变较明显，应先用多个已知刻度点求取平面单应矩阵或一维分段映射，再计算球道坐标。设摆杆在图像中的有效长度为  $W$  像素，则理想空间分辨率约为  $L/W$ ；为满足  $\pm 1$  cm 的控制误差，视觉静态标定残差宜控制在 2 mm 以内。

钢球速度不能按固定帧率简单差分，而应使用相邻有效帧的真实时间间隔。为抑制像素噪声，可采用带低通的差分估计：

$$v_k = \alpha v_{k-1} + (1 - \alpha) \frac{x_k - x_{k-1}}{t_k - t_{k-1}}, \quad 0 \leq \alpha < 1 \quad (6)$$

$\alpha$  过大会使速度曲线平滑但增加相位滞后， $\alpha$  过小则会放大定位抖动，因此需要结合相机帧率和实测噪声整定。

### 2.3、球位置串级闭环控制原理

球位置控制采用外慢内快的串级结构。外环被控量为钢球位置  $x$ ，输入为目标位置  $x_{\text{ref}}$ ；位置误差经过比例环节产生纠偏角度，钢球速度反馈提供阻尼，以减小往返振荡。外环可先采用 PD 形式：

$$\theta_{\text{fb}} = K_{px}(x_{\text{ref}} - x) - K_{dx}v \quad (7)$$

在此基础上叠加由底盘任务状态产生的  $\theta_{\text{task\_ff}}$  和由 BMI088 纵向加速度产生的  $\theta_{\text{accel\_ff}}$ ，得到摆杆目标角：

$$\theta_{\text{ref}} = \text{sat}(\theta_{\text{fb}} + \theta_{\text{task\_ff}} + \theta_{\text{accel\_ff}}) \quad (8)$$

其中  $\text{sat}$  表示角度、角速度和变化率限制。任务前馈用于加速、制动、转弯等可预知阶段，加速度前馈用于补偿实际快速扰动；二者均不计算  $x_{\text{ref}} - x$ ，因此不能替代视觉反馈。任何前馈异常时均应在有限时间内渐退为零，视觉闭环仍保持独立可用。

摆杆内环使用 QD4310 反馈的实际角度  $\theta$ 、角速度  $\omega$  及电流信息。控制电流可写为：

$$I_{\text{cmd}} = \text{sat}[K_{p\theta}(\theta_{\text{ref}} - \theta) - K_{d\theta}\omega + I_0] \quad (9)$$

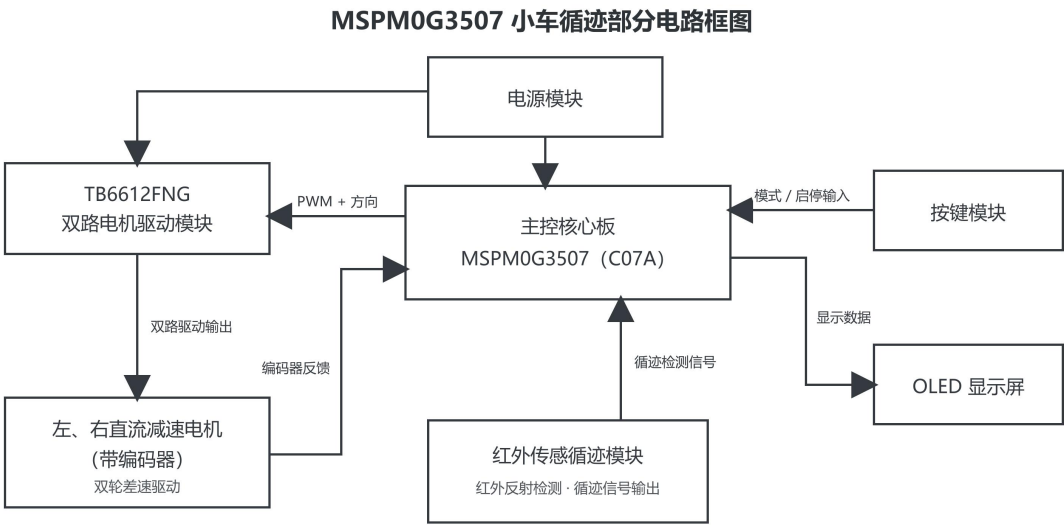
$I_0$  用于补偿机构静态偏置，机械配平良好时可设为零。摆杆内环应快于钢球位置外环，调试时依次调整位置比例、速度阻尼和积分参数；使用积分控制时应加入抗饱和处理。

结论：系统采用钢球位置外环、摆杆伺服内环及加速度前馈的分层控制结构。MSPM0G3507、树莓派 5B 和大疆 C 板分别负责底盘控制、视觉处理和滚球控制，具有较好的实时性、抗干扰能力和安全性。

3、电路与程序设计

3.1、电路设计

3.1.1、循迹小车总体电路

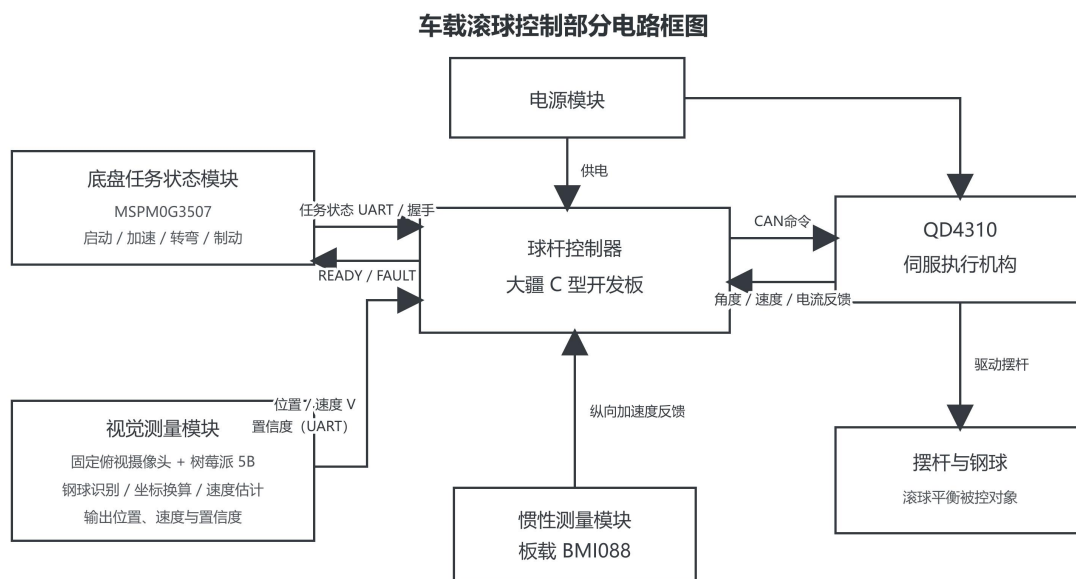


图三：小车循迹部分电路框图

3.1.2、平衡滚球控制电路

滚球控制系统以 RoboMaster 开发板 C（STM32F407）为主控，连接树莓派视觉模块、QD4310 执行机构和板载 BMI088。控制任务每 5 ms 执行一次，负责电机命令发送和反馈处理。

树莓派通过 115200 bit/s 的 UART 向 C 板发送钢球位置和速度数据；QD4310 通过 1 Mbit/s 的 CAN 总线与 C 板通信；BMI088 通过 SPI1 采集车体加速度。摆杆水平角设为 1.2461 rad，正常工作范围为 1.10~1.40 rad，电机电流和速度分别限制为 1.65 A 和 1000 r/min。系统每次上电后均需检查电机配置。



图四：车滚球控制部分电路

### 3.2 设计思路

#### 3.2.1 循迹小车程序设计总体思路

循迹小车以 MSPM0G3507 为主控制器。红外循迹模块检测轨迹偏差，主控采用差速控制计算左右轮目标速度，并根据编码器反馈进行 PID 调节，通过 TB6612FNG 驱动电机。程序每 10 ms 执行一次循迹和速度控制，主循环负责按键、模式切换和 OLED 显示，状态机统一管理启动、运行和停车过程。

#### 3.2.2 车载滚球控制装置

车载滚球控制装置以大疆 C 型开发板为核心。树莓派 5B 处理摄像头图像，并通过 UART 发送钢球的位置和速度信息；BMI088 检测车体加速度；MSPM0G3507 发送车辆任务状态。C 板综合任务状态和传感器数据计算摆杆目标角度，再通过 CAN 总线控制 QD4310 执行机构，使钢球保持平衡。程序采用状态机管理数据接收、平衡控制和故障停车过程。

### 4.测试方案和测试结果

#### 4.1 测试方案

##### 4.1.1 循迹部分硬件测试

检查主控板、电机驱动、电机、编码器、红外传感器、按键和 OLED 的连接及共地情况。上电后依次测试供电、显示、传感器状态、电机转向、PWM 调速和编码器反馈，确认正常后进行地面测试。

#### 4.1.2 软件仿真测试

利用 CCS 观察程序变量，检查 10 ms 控制中断、红外数据读取、偏差计算、目标速度、PID 输出和模式切换。分别模拟全白、居中、左偏、右偏和全黑状态，验证三种运行模式的控制时序。

#### 4.1.3 硬件软件联调

先在车轮架空状态下调试速度闭环，再在实际赛道测试三种模式。记录传感器状态、循迹偏差、车轮速度和 PWM 输出，比较运行时间、偏离距离和停车位置。

#### 4.2 测试条件与仪器

测试时控制周期设为 10 ms，各模块按原理图连接并可靠共地，赛道和环境光照保持一致。测试仪器包括直流稳压电源、数字万用表、示波器、计算机、CCS 调试器、秒表和量具，分别用于供电、波形、程序变量、运行时间及偏离距离测量。

#### 4.3 测试结果及分析

##### 4.3.1 测试结果（数据）

当前工程的关键参数及静态核验结果如下表所示：（速度单位/rps，时间单位/s）

表一

| 测试项目 | 控制周期   | 模式 1 速度 | 模式 2 速度   | 模式 3 速度  | PID 输出 | 循迹阈值     |
|------|--------|---------|-----------|----------|--------|----------|
| 代码设定 | 10 ms  | 2.20    | 1.77      | 1.70     | ±8000  | 800/1300 |
| 核验结果 | 100 Hz | 1 s 后阶跃 | 28.6 s 停车 | 9.0 s 停车 | 限幅有效   | 带滞回判断    |

##### 4.3.2 测试分析与结论

测试结果表明，程序已实现红外循迹、编码器测速、左右轮速度闭环和三种运行模式。循迹偏差由八路传感器加权计算，黑线和白底阈值分别为 800 和 1300。速度 PID 参数为  $K_p=2000$ 、 $K_i=200$ 、 $K_d=5$ ，输出限幅为 ±8000。

控制程序每 10 ms 执行一次速度计算、循迹处理和电机闭环控制。模式 2 采用缓慢加速、匀速和减速运行，总时间约 28.6 s；模式 3 采用快速加速、匀速和减速运行，总时间约 9.0 s；模式 1 直接启动，检测到终点后延时 500 ms 停车。

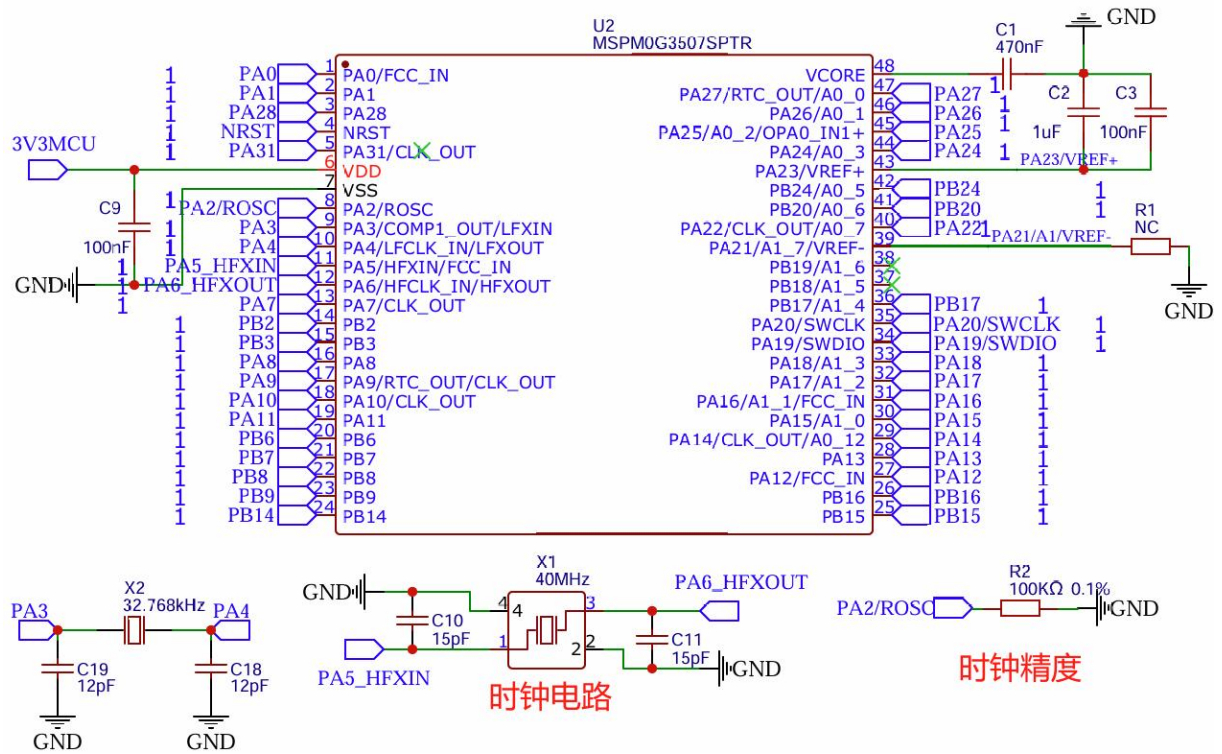
综上，当前程序能够完成基本循迹、速度闭环以及不同模式下的定时变速和停车控制。

---

## 五、参考文献

- [1] 吴运昌。模拟电子线路基础 [M]. 广州：华南理工大学出版社，2004.
- [2] 李建忠。单片机原理及应用 [M]. 西安：西安电子科技大学出版社，2002.
- [3] 阎石。数字电子技术基础 [M]. 北京：高等教育出版社，1997.
- [4] 候正信，王安国。数字信号处理基础 [M]. 北京：电子工业出版社.
- [5] 张晓丽，等。数据结构与算法 [M]. 北京：机械工业出版社，2002.
- [6] 童诗白。模拟电子技术基础（第四版）[M]. 北京：高等教育出版社.
- [7] TEXAS INSTRUMENTS. [MSPM0G350x Mixed-Signal Microcontrollers With CAN-FD Interface: SLASEX6C](#)[EB/OL]. 2025[2026-08-01].
- [8] 深圳市大疆创新科技有限公司. RoboMaster 开发板 C 型用户手册[Z]. 2020.

## 附录 1：电路原理图



## 附录 2 MSPM0G3597 代码框架图

```
void Control_Update(void)
{
    float tl=0,tr=0,al,ar,p1,pr,e,b;
    Update_Gray_Sensors();
    al=Get_Speed_Left();
    ar=Get_Speed_Right();
    if(!motor_started||stop_flag){
        Set_PWM(0,0);startup_cycle=0;ctrl_state=TRACKING;return;
    }
    switch(ctrl_state){
    case TRACKING:
        if(!line_found){Set_PWM(0,0);startup_cycle=0;return;}
        e=(float)line_error/320.0f;
        if(e>1)e=1;if(e<-1)e=-1;
        b=(line_confidence<25)?0.8f:BASE_RPS;
        tl=b+e;tr=b-e;
        if(startup_cycle<SOFT_START_CYCLES){
            e=(float)startup_cycle++/SOFT_START_CYCLES;
            tl*=e;tr*=e;
        }
        break;
    case WHITE_HOLD:
        tl=hold_speed_left;tr=hold_speed_right;
        if(++hold_cycles>=hold_target)ctrl_enter_state(LEFT_TURN);
        break;
    case LEFT_TURN:
        tl=0;tr=TURN_SPEED_B;
        if(get_black_count(>0)ctrl_enter_state(TRACKING);
        break;
    default:Set_PWM(0,0);return;
    }
    p1=PID_Update(&pid_left,tl,al,DT);
    pr=PID_Update(&pid_right,tr,ar,DT);
    if(fabs(p1)>ENC_FAULT_PWM&&fabs(al)<ENC_FAULT_SPEED)p1*=0.3f;
    if(fabs(pr)>ENC_FAULT_PWM&&fabs(ar)<ENC_FAULT_SPEED)pr*=0.3f;
    Set_PWM((int)p1,(int)pr);
}
```

### 附录 3 大疆 c 板代码框架图

BallControl\_Update.cpp

```
1 void BallControl_Update(void)
2 {
3     QD4310::Snapshot motorData = motor.snapshot();
4     VisionLinkSnapshot vision();
5     VisionLink_GetSnapshot(&vision);
6
7     bool motorOk = motorData.hasFeedback
8         && isfinite(motorData.angleRad)
9         && isfinite(motorData.speedRpm)
10        && isfinite(motorData.currentA);
11
12     bool visionOk = vision.valid
13         && vision.measurement_age_ms <= 80
14         && vision.confidence >= 180;
15
16     if (EmergencyStop() || !motorOk || CanFault()) {
17         controlState = FAULT;
18         motor.setSpeed(0.0f);
19         motor.setCurrent(0.0f);
20         motor.disable();
21         return;
22     }
23
24     float angleOffsetDeg = 0.0f;
25
26     switch (controlState) {
27     case WAIT_MOTOR:
28         if (MotorFeedbackCount() >= 5)
29             controlState = ENABLE_PENDING;
30         break;
31
32     case ENABLE_PENDING:
33         motor.enable();
34         if (MotorEnableConfirmCount() >= 3)
35             controlState = LEVEL_HOLD;
36         break;
37
38     case LEVEL_HOLD:
39         if (visionOk && NewVisionCount() >= 10)
40             controlState = BALANCING;
41         break;
42
43     case BALANCING:
44         if (!visionOk) {
45             controlState = VISION_RECOVERY;
46             break;
47         }
48
49         float errorMm = (vision.target_position_0p1mm
50             - vision.position_0p1mm) * 0.1f;
51         float velocityMmps = vision.velocity_valid
52             ? vision.velocity_mmps : 0.0f;
53
54         angleOffsetDeg = 0.25f * errorMm
55             - 0.15f * velocityMmps;
56
57         float limit = (vision.task_id == 3) ? 4.0f : 3.0f;
58         angleOffsetDeg = Clamp(angleOffsetDeg, -limit, limit);
59         angleOffsetDeg += GetImuFeedforward();
60         break;
61
62     case VISION_RECOVERY:
63         if (visionOk && NewVisionCount() >= 3)
64             controlState = BALANCING;
65         break;
66
67     default:
68         motor.disable();
69         return;
70     }
71
72     float targetAngleRad = 1.2461f
73         + angleOffsetDeg * PI / 180.0f;
74     targetAngleRad = SlewLimit(targetAngleRad, 25.0f, 0.005f);
75     motor.setAngle(targetAngleRad);
76 }
```

---